Homework Assignment: Multilayer Perceptron (MLP)

1 Learning Objectives

Upon completing this assignment, you should be able to:

- understand MLP architecture, forward propagation, cost functions, backpropagation, weight initialization, and parameter-update strategies;
- apply an MLP to a biological classification task and evaluate its predictive performance critically.

2 Problems

2.1 Problem 1: Understanding MLP architecture (8 points)

Consider an MLP designed to predict whether a patient has a disease using measurements of 10 metabolites. The network has the following architecture:

$$\underbrace{10}_{\text{input features}} \longrightarrow \underbrace{8}_{\text{hidden layer 1}} \longrightarrow \underbrace{4}_{\text{hidden layer 2}} \longrightarrow \underbrace{1}_{\text{output}}.$$

All successive layers are fully connected; each non-input neuron has a bias.

a. How many layers does the network have, counting the input layer?

b. How many trainable weights and biases are there in each non-input layer?

c. Calculate the total number of trainable parameters.

d. Explain why nonlinear activation functions are necessary in the hidden layers.

2.2 Problem 2: Forward propagation by hand (12 points)

Consider a network with two inputs, two hidden neurons, and one output neuron. The hidden layer uses ReLU, and the output layer uses sigmoid. For a single sample,

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad W^{(1)} = \begin{bmatrix} 0.5 & -0.2 \\ 0.3 & 0.4 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix},$$

$$W^{(2)} = \begin{bmatrix} 0.8 & -0.5 \end{bmatrix}, \qquad b^{(2)} = 0.2.$$

Use

$$\mathbf{z}^{(1)} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}, \quad \mathbf{a}^{(1)} = \mathrm{ReLU}(\mathbf{z}^{(1)}),$$

$$z^{(2)} = W^{(2)}\mathbf{a}^{(1)} + b^{(2)}, \quad \hat{y} = \sigma(z^{(2)}), \qquad \sigma(z) = \frac{1}{1+e^{-z}}.$$

a. Calculate the hidden-layer pre-activation vector $\mathbf{z}^{(1)}$.

b. Calculate the hidden-layer activation vector $\mathbf{a}^{(1)}$.

c. Calculate the output pre-activation $z^{(2)}$.

d. Calculate the predicted probability $\hat{y}$.

e. Calculate the binary cross-entropy loss for $y = 1$:

$$J = -\left[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\right].$$

Show calculations. A calculator or Python may be used to verify numerical results.

2.3 Problem 3: Backpropagation using the chain rule (15 points)

Consider a simplified neural network with one input, one hidden neuron, and one output neuron. Both neurons use sigmoid activation:

$$z_1 = w_1 x + b_1, \qquad a_1 = \sigma(z_1),$$

$$z_2 = w_2 a_1 + b_2, \qquad \hat{y} = \sigma(z_2).$$

For one sample, the binary cross-entropy cost is

$$J = -\left[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\right].$$

Given $x = 2$, $y = 1$, $w_1 = 0.5$, $b_1 = 0$, $w_2 = 1$, and $b_2 = 0$:

a. Calculate z_1, a_1, z_2, $\hat{y}$, and J.

b. Using the chain rule, derive $\dfrac{\partial J}{\partial z_2} = \hat{y} - y$.

c. Derive expressions for $\dfrac{\partial J}{\partial w_2}$ and $\dfrac{\partial J}{\partial b_2}$.

d. Derive expressions for $\dfrac{\partial J}{\partial w_1}$ and $\dfrac{\partial J}{\partial b_1}$.

e. Calculate all four gradients numerically.

2.4 Problem 4: One step of gradient descent (10 points)

Using the gradients from Problem 3 and learning rate $\alpha = 0.1$:

a. Update all four parameters *simultaneously* using $\theta_{\text{new}} = \theta_{\text{old}} - \alpha\, \partial J / \partial \theta$.

b. Perform forward propagation with the updated parameters.

c. Calculate the new BCE loss.

d. Did the loss decrease? Explain why gradient descent does not necessarily decrease the loss at every update for arbitrary learning rates.

2.5 Problem 5: Weight initialization (8 points)

An MLP has 20 input features, a hidden layer with 10 ReLU neurons, and an output layer with one sigmoid neuron.

a. Explain the symmetry problem caused by initializing all hidden-layer weights to zero.

b. Why can biases generally be initialized to zero?

c. Calculate the recommended initial weight variance for the hidden layer using He initialization:

$$\mathrm{Var}(W) = \frac{2}{n_{\text{in}}}.$$

d. Calculate the recommended initial weight variance for the same layer using Xavier initialization:

$$\mathrm{Var}(W) = \frac{2}{n_{\text{in}} + n_{\text{out}}}.$$

e. Explain why He initialization is often preferred for ReLU hidden layers.

2.6 Problem 6: Batch size and parameter updates (7 points)

Suppose a biological dataset contains 1,000 training samples. An MLP is trained for 50 epochs.

a. Calculate the number of parameter updates per epoch and across all 50 epochs for each strategy:

- Full-batch gradient descent;
- Stochastic gradient descent (SGD);
- Mini-batch gradient descent with batch size 32.

b. Explain why SGD produces more variable parameter updates than full-batch gradient descent.

c. Discuss two advantages of mini-batch gradient descent.

Assume that the final incomplete mini-batch is included.

2.7 Problem 7 Breast cancer classification using MLP (40 points)

Biological objective. Predict whether a breast tumor is malignant or benign using quantitative measurements derived from digitized images of breast cell nuclei. Use the Wisconsin Diagnostic Breast Cancer dataset available in `sklearn.datasets`. It contains 569 samples and 30 numerical features. This is a manageable biological classification example, not a gene-expression dataset.

Task A. Data exploration and preprocessing (6 points)

Load the dataset:

```
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()
X = data.data
y = data.target
```

a. Report the number of samples, features, and samples in each class.

b. Explain what the input features represent biologically.

c. Split the data into 80% training and 20% test samples using `random_state=42` and stratification.

d. Standardize features using `StandardScaler`. Explain why the scaler must be fitted only on training data.

Task B. Train logistic regression and MLP (8 points)

Train two classifiers using identical training and test sets:

- Logistic regression as a baseline.
- An MLP with one hidden layer containing 16 neurons, ReLU activation, and `random_state=42`.

Use scikit-learn pipelines to prevent preprocessing leakage. Start with:

```
from sklearn.neural_network import MLPClassifier

MLPClassifier(
    hidden_layer_sizes=(16,),
    activation="relu",
    solver="adam",
    alpha=0.001,
    max_iter=2000,
    random_state=42
)
```

For both models, calculate test-set accuracy, precision, recall, F1 score, and ROC AUC. Present results in a comparison table. Specify which class is treated as positive when computing precision, recall, and F1.

Task C. Investigate MLP architecture (8 points)

Compare the following architectures, keeping other settings fixed:

Model	Hidden-layer sizes
MLP-1	`(8,)`
MLP-2	`(16,)`
MLP-3	`(32,)`
MLP-4	`(32, 16)`

Use five-fold stratified cross-validation on the *training data* to compare mean ROC AUC. Select the best architecture using cross-validation results, refit it on the full training set, and evaluate it once on the held-out test set. Explain whether increasing the number of neurons or hidden layers consistently improves predictive performance. Include scaling inside the cross-validation pipeline.

Task D. Analyze learning behavior (8 points)

For the selected MLP:

a. Plot training loss versus iteration using the fitted model's `loss_curve_` attribute.

b. Describe whether training loss appears to converge.

c. Explain how a learning rate that is too large or too small might affect the curve.

d. Compare the MLP with logistic regression. Does the MLP provide a meaningful improvement? Explain why a more complex model might not outperform a simpler model on this dataset.

3 Submission Requirements

Submit a single Jupyter notebook (`.ipynb`) containing written mathematical derivations, Python code, numerical results, figures, and interpretations. Handwritten derivations may be attached as a separate PDF if necessary. Clearly distinguish model selection using cross-validation from final evaluation using the held-out test set. Include all code needed to reproduce your results and set random seeds where appropriate.